



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

API INTEGRATION GUIDE

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	API INTEGRATION GUIDE
Version	v1.0.0
Date	02 July 2026

How the Frontend Connects to the Backend

1. Architecture Overview

```

React Component
  ↓ calls
Service Function (e.g. medicineService.search)
  ↓ calls
axiosClient.get('/medicines/search', { params })
  ↓ attaches JWT token automatically
HTTP Request → FastAPI Backend
  ↓
HTTP Response
  ↓ interceptors check for 401
Service receives response.data
  ↓
Component updates state with data
  ↓
React re-renders UI

```

2. Axios Client (`src/config/axiosClient.js`)

The Axios client is the single HTTP gateway for the entire frontend.

Configuration

```

const axiosClient = axios.create({
  baseURL: env.API_BASE_URL,           // http://localhost:8000/api/v1
  headers: { 'Content-Type': 'application/json' },
  timeout: 15000,                      // 15 second timeout
})

```

Request Interceptor — JWT Token

Every outgoing request automatically gets the JWT Bearer token:

```

axiosClient.interceptors.request.use((config) => {
  const token = storage.get(STORAGE_KEYS.ACCESS_TOKEN)
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

```

You never manually add `Authorization` headers in services — it happens automatically.

Response Interceptor — 401 Handling

When any request returns 401 Unauthorized (expired token):

```

axiosClient.interceptors.response.use(
  (response) => response,
  (error) => {

```

```

    if (error.response?.status === 401) {
      // Clear tokens from localStorage
      storage.remove(STORAGE_KEYS.ACCESS_TOKEN)
      storage.remove(STORAGE_KEYS.REFRESH_TOKEN)
      storage.remove(STORAGE_KEYS.USER)

      // Redirect to session-expired page
      window.location.href = ROUTES.SESSION_EXPIRED
    }
    return Promise.reject(error)
  }
}
)

```

Future enhancement: Replace the redirect with a token refresh flow (see [axiosClient.js](#) JSDoc comment).

3. Service Layer (`src/services/`)

Why Services Exist

Without services:

```

// ■ Bad – API call directly in component
function MedicineDetailsPage() {
  useEffect(() => {
    axios.get(`http://localhost:8000/api/v1/medicines/${id}`)
      .then(res => setMedicine(res.data))
  }, [id])
}

```

Problem: URL hardcoded in component. When backend changes, must find/replace everywhere.

With services:

```

// ■ Good – component calls service
function MedicineDetailsPage() {
  useEffect(() => {
    medicineService.getById(id).then(res => setMedicine(res.data))
  }, [id])
}

// In medicineService.js – only one place to update
getById: (id) => axiosClient.get(`/medicines/${id}`)

```

All Services

[authService.js](#)

```

authService.register(data)           // POST /auth/register
authService.login(credentials)       // POST /auth/login
authService.verifyOtp(data)          // POST /auth/verify-otp
authService.resendOtp(data)          // POST /auth/resend-otp
authService.forgotPassword(data)     // POST /auth/forgot-password
authService.resetPassword(data)      // POST /auth/reset-password
authService.refreshToken(data)       // POST /auth/refresh-token
authService.logout()                 // POST /auth/logout

```

[medicineService.js](#)

```

medicineService.search(params)      // GET /medicines/search?q=...
medicineService.getById(id)         // GET /medicines/:id
medicineService.getAlternatives(id) // GET /medicines/alternatives/:id
medicineService.getAll(params)      // GET /medicines
medicineService.create(data)        // POST /medicines
medicineService.update(id, data)    // PUT /medicines/:id
medicineService.remove(id)          // DELETE /medicines/:id

```

pharmacyService.js

```

pharmacyService.getNearby(params)    // GET /pharmacies/nearby?lat=...&lng=...
pharmacyService.search(params)       // GET /pharmacies/search?q=...
pharmacyService.getById(id)          // GET /pharmacies/:id
pharmacyService.getInventory(id, params) // GET /pharmacies/:id/inventory
pharmacyService.create(data)         // POST /pharmacies
pharmacyService.update(id, data)     // PUT /pharmacies/:id

```

userService.js

```

userService.getMe()                  // GET /users/me
userService.updateMe(data)           // PUT /users/me
userService.deleteMe()               // DELETE /users/me
userService.getAllUsers(params)      // GET /users (admin only)
userService.getUserById(id)          // GET /users/:id (admin only)

```

inventoryService.js

```

inventoryService.getAll(params)      // GET /inventory
inventoryService.create(data)        // POST /inventory
inventoryService.update(id, data)    // PUT /inventory/:id
inventoryService.remove(id)          // DELETE /inventory/:id

```

4. TanStack React Query (`src/config/queryClient.js`)

React Query is a server-state management library. It handles:

- Caching API responses
- Background re-fetching
- Loading/error states
- Deduplication of identical requests

Configuration

```

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60,          // Data fresh for 1 minute
      gcTime: 1000 * 60 * 5,        // Cache kept for 5 minutes
      retry: 1,                      // Retry failed requests once
      refetchOnWindowFocus: false,  // Don't re-fetch on tab focus
    }
  }
})

```

Usage Pattern (when backend is ready)

```

import { useQuery } from '@tanstack/react-query'
import medicineService from '../services/medicineService'

```

```
function MedicineDetailsPage() {
  const { id } = useParams()

  const { data, isLoading, isError, refetch } = useQuery({
    queryKey: ['medicine', id],
    queryFn: () => medicineService.getById(id).then(res => res.data),
  })

  if (isLoading) return <Skeleton />
  if (isError) return <ErrorState onRetry={refetch} />

  return <div>{data.name}</div>
}
```

Why React Query instead of useState + useEffect?

- Automatic caching — visit the same medicine page twice: second time is instant
- Automatic loading/error states — no boilerplate `useState(false)` for `isLoading`
- Background updates — data refreshes silently while users read it
- Query invalidation — after adding inventory, the list auto-refreshes

5. HTTP Methods Explained

Method	Purpose	When Used
GET	Retrieve data (no side effects)	Load pages, search, fetch details
POST	Create new data	Register, login, add medicine, submit form
PUT	Replace existing data completely	Update profile, edit inventory
PATCH	Update part of existing data	Mark notification as read, change status
DELETE	Remove data	Delete user, delete stock entry

6. Complete Request Example — Medicine Search

Frontend code

```
// In MedicineSearchPage, using useDebounce to avoid excessive API calls
const debouncedQuery = useDebounce(searchQuery, 400)

const { data, isLoading } = useQuery({
  queryKey: ['medicines', 'search', debouncedQuery],
  queryFn: () => medicineService.search({ q: debouncedQuery, limit: 10 }),
  enabled: debouncedQuery.length >= 2, // Only search if 2+ characters
})
```

What happens step by step

1. User types "parace" in the search input
2. `useDebounce` waits 400ms after user stops typing
3. React Query fires the query (`queryFn`)
4. `medicineService.search({ q: 'parace', limit: 10 })` is called
5. `axiosClient.get('/medicines/search', { params: { q: 'parace', limit: 10 } })` fires

6. JWT token is attached by the request interceptor
 7. Request: `GET http://localhost:8000/api/v1/medicines/search?q=parace&limit=10`
 8. Response comes back with medicine array
 9. React Query caches the result under key `['medicines', 'search', 'parace']`
 10. `data` in the component updates, React re-renders with results
-

7. Complete Request Example — Login

Frontend AuthContext.login()

```
// Current (mock – for frontend-only development):
const data = {
  user: { id: '1', name: 'Demo User', email: credentials.email, role: 'patient' },
  accessToken: 'mock_access_token',
  refreshToken: 'mock_refresh_token',
}

// Future (when backend ready – uncomment this line):
// const { data } = await authService.login(credentials)
```

What to do when backend is ready

11. Open `src/contexts/AuthContext.jsx`
 12. Find the `login` function
 13. Remove the mock data block
 14. Uncomment: `const { data } = await authService.login(credentials)`
 15. Done — no other changes needed
-

8. Environment Configuration

Set the API URL in `.env`:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

For production:

```
VITE_API_BASE_URL=https://api.smartmedicine.example.com/api/v1
```

`env.js` reads this:

```
const env = {
  API_BASE_URL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000/api/v1',
}
```

`axiosClient.js` uses it as `baseURL`.

9. Error Handling Pattern

```
// Service call with proper error handling:
const login = useCallback(async (credentials) => {
  setIsLoading(true)
  setAuthError(null)
```

```

try {
  const { data } = await authService.login(credentials)
  storeSession(data.user, data.accessToken, data.refreshToken)
  return data.user
} catch (err) {
  // err.response.data.message comes from FastAPI backend
  const msg = err?.response?.data?.message ?? 'Login failed. Please try again.'
  setAuthError(msg)
  throw err // Re-throw so the page form can also react
} finally {
  setIsLoading(false) // Always runs – clears loading state
}
}, [])

```

10. JWT Token Flow

1. User logs in → POST /auth/login
2. Backend returns { accessToken, refreshToken }
3. Frontend stores both in localStorage (storage.js)
4. Every request: axiosClient adds "Authorization: Bearer {accessToken}"
5. Token expires (e.g. 1 hour)
6. Request returns 401 Unauthorized
7. axiosClient interceptor catches it
8. CURRENT: clears session → redirects to /session-expired
9. FUTURE: POST /auth/refresh-token → get new accessToken → retry original request